Janiya Green

002831712

Computer Science

Prompt 1 / Failure Reconstruction

Use your T1/T4 observations: imagine a classmate can see three cards after adding them, but all three appear under Harbor after reopening—even though one was saved to Atlas—and Part I guest rows remain visible. Give two distinct causes in the write/query/UI chain, one observation that would distinguish them, and the safest next diagnostic action. Explain how the saved folder_id and the list query should be checked without changing data. Do not erase or reseed data as a first response.

- Cause 1: the card may have been saved with the wrong folder_id, causing it to belong to Harbor instead of Atlas.
- Cause 2: the folder screen may be displaying every card instead of filtering cards by the selected folder
- One observation that would distinguish them: Compare the card's saved folder_id with Atlas's ID. If it matches Atlas, the problem is probably in the query or display logic. If it matches Harbor, the problem likely happened when saving the card.
- Safest Next Diagnostic Action: Use a read-only database query to check the folder IDs and inspect the code that loads cards. Check whether the query uses a condition such as WHERE folder_id = ?.

Prompt 2 / Delete Policy Trade-Off

Your schema uses ON DELETE CASCADE for folders and cards. Consider a clinic appointment system where deleting a patient must not silently erase billing/audit records. Specify a better delete policy, one UI safeguard, and one test that would demonstrate it. Tie the comparison back to the actual Atlas/Harbor behavior you observed and identify which child records the cascade removed.

- Better delete policy: Use Soft deletion to mark a patient as inactive without permanently removing their information.

- One UI Safeguard: Show a confirmation message explaining what will happen before deletion. If related records prevent deletion, display a message explaining why the patient must remain in the system.
- 1 test that demonstrates:
    - Creating a patient with billing records, attempting to delete them, and verifying that the deletion is blocked and the records remain.
    - With the Atlas/Harbor folders, the cascade policy automatically removes cards belonging to a deleted folder. Deleting Atlas would remove all its associated cards, while Harbor and its cards should remain. This shows why automatic deletion can be risky when records must be preserved.

Prompt 3 / Image Forensics

After T4, Harbor's Signal card and its notes are present, but its artwork is missing. Construct a short diagnostic decision tree that distinguishes a missing bundled asset, deleted app-private file, unavailable URL, and malformed reference. State what should remain visible, which data should not be deleted, and what your null-reference T6 fallback proves. Include your chosen storage approach and its trade-off.

- Short Diagnostic Tree:
    - First, check whether image_ref is empty or incorrectly formatted. If it is a bundled asset, confirm that the file exists and is listed in pubspec.yaml. If it is a private file, check whether the file still exists on the device. If it is a URL, check whether the address is valid and the image can load.
- What should stay visible: The Signal card's title, notes, suit, and other saved information should still appear even if its artwork cannot load. Do not delete the card, its notes, or id just because the image is missing.
- T6 Fallback: If a card with a null image reference displays a default suit symbol, it proves the app can handle a missing image reference without crashing or removing the card. It does not prove that a missing asset file or unavailable URL will load successfully.
- Chosen Storage and Tradeoff:
    - Store an optional image URL in image_ref and show a default suit symbol when the image is unavailable.
    - This keeps the database small and makes images easy to change, but the artwork depends on the URL continuing to work and being accessible.

The AI Alchemist's Lab

U1: The twin-card identity trick

Claim: “A title is unique enough to edit a card.” Use the two Compass cards in Atlas, or create two fictional cards with the same title but distinct IDs. How would you determine which row an edit should change? Use your own IDs and a query or source trace to compare title-based and ID-based targeting, without executing a destructive title-based update. What surprising user action would expose the flaw, and how would you test your fix?

- Predict
    - Using a card title to identify a record for editing is unsafe because two cards can have the same title. In my card catalogue, Atlas can contain two cards titled Compass even though they represent separate database records. The title describes the card, but it does not uniquely identify a particular row. I expect an ID-based edit to target one specific card while title-based targeting could match both records.
- Investigate
    - I used the Atlas folder and its two Compass cards as the test scenario. The relevant evidence is T2, which checks the saved card records, and T3, which checks whether editing one card changes only the selected record.
- Challenge
    - The evidence can show whether titles are duplicated and whether the application's current edit operation targets a specific record. However, finding two matching titles does not, by itself, prove that the application actually updates both. The result also depends on the SQL WHERE condition and the code that passes the selected card's ID to the database helper.
- Recast the spell
    - The application should use the card's primary key, id, for editing and deleting records. The selected card's ID should be passed into the database helper, and the SQL update should use WHERE id = ?. The app should also check the number of affected rows and report if the selected record no longer exists.

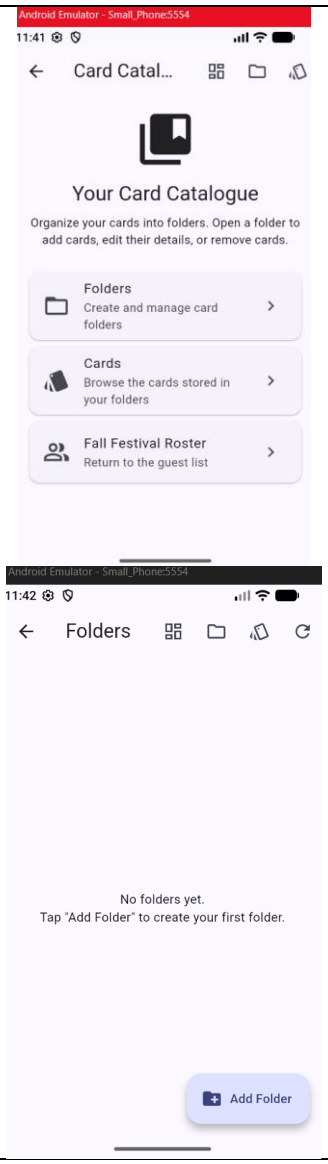
U3: The vanishing artwork spell

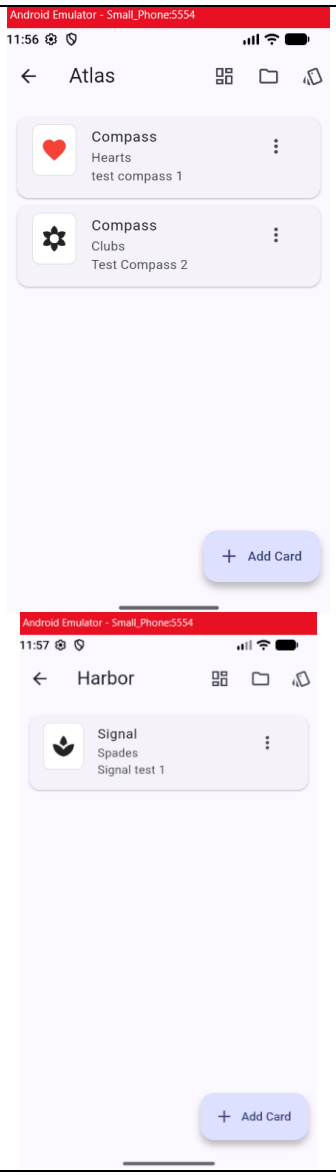
Claim: “Saving an image URL makes the picture available offline.” Use your chosen image-reference approach and T6 evidence to distinguish a saved reference from available bytes. Predict what happens without network access, explain what your observation can actually

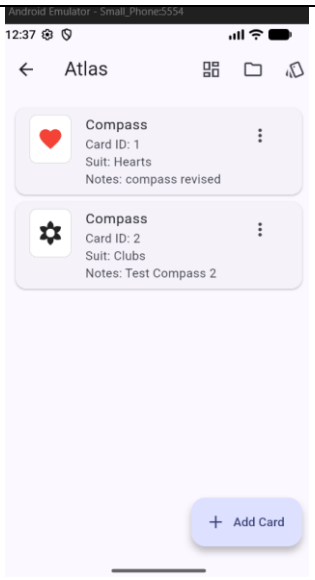
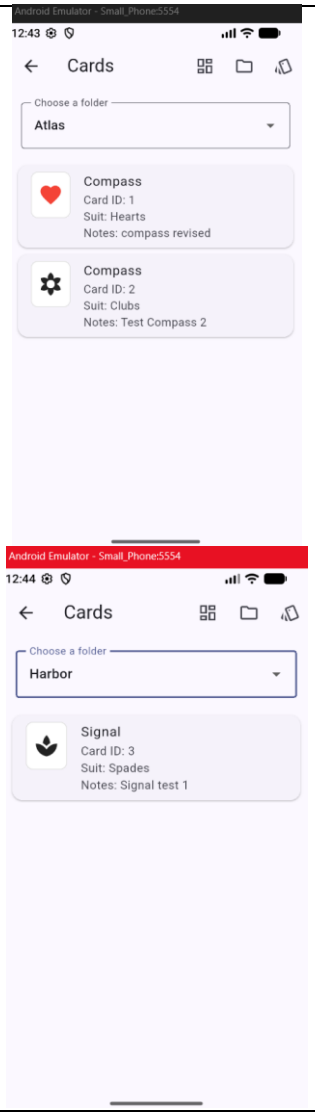
prove, and propose a fallback that preserves the card's identity. What test would distinguish a cached image from a genuinely offline asset?

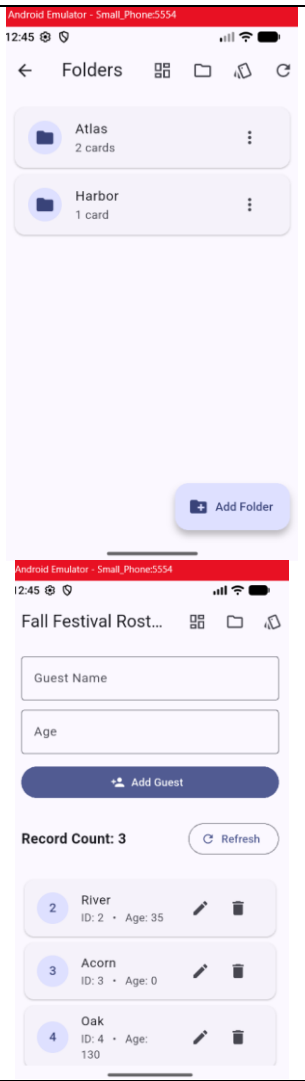
- Predict
    - Saving an image URL in the database will preserve the reference, but will not guarantee that the actual image is available without an internet connection. A URL identifies where the app can try to retrieve an image. It does not necessarily store the image bytes on the device. I expect a card with an unavailable URL to retain its title and notes while its artwork fails to load and a fallback symbol appears, provided the app handles image-loading errors.
- Investigate
    - My implementation accepts an optional image reference and stores it in the card's image_ref field. In CardArtwork, the app checks whether the reference begins with https:// or http://. If it does, the widget attempts to display the image with Image.network. Its errorBuilder returns the fallback artwork if loading fails. If the reference is null, blank, or not an HTTP(S) URL, the widget displays the fallback instead.
- Challenge
    - A successful database query proves that the image reference was stored; it does not prove that the image itself was downloaded or can be displayed offline. A failed network image could be caused by a disabled connection, an unavailable server, a deleted remote image, an invalid URL, or a temporary network failure. The T6 null-reference fallback can demonstrate how the app behaves when it has no usable image reference, but it cannot prove that a previously loaded image is cached or that the app works offline.
- Recast the spell
    - I would keep the URL approach but treat it as a remote reference rather than an offline copy. The app should preserve the card's database ID, title, suit, notes, and folder assignment even if artwork cannot be loaded. A fallback suit symbol should communicate that the card still exists even when its image is unavailable.

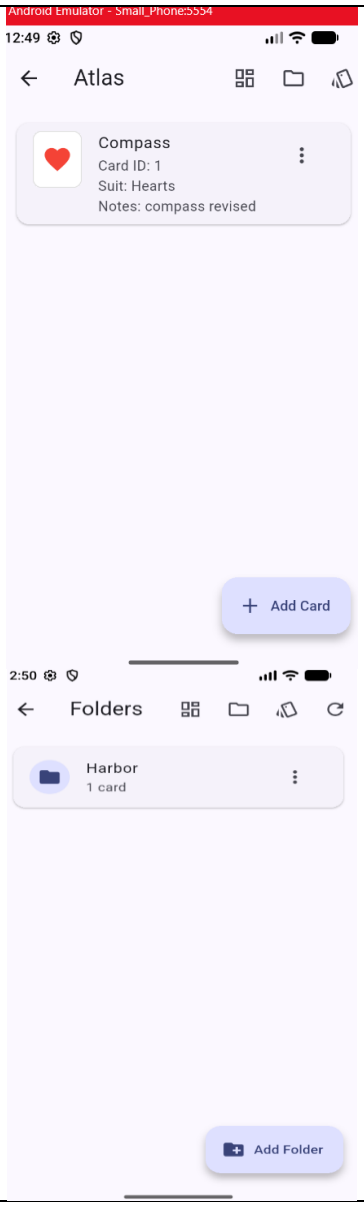
Test	Action/input	Expected Observation	Actual Observation	Pass/Fail

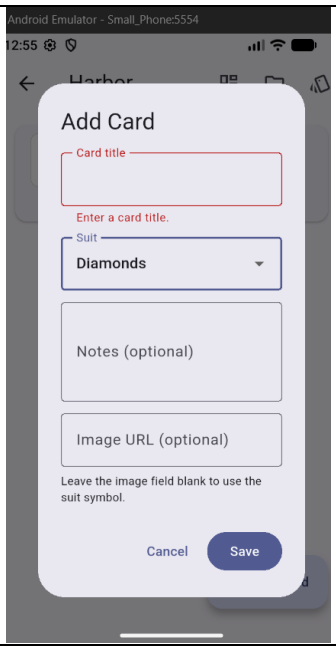
T1 / Upgrade & fresh creation	T1a: open the existing version-1 install with the new helper. T1b: launch version 2 on a separate disposable fresh target.	T1a retains all original IDs/values and creates the new tables/index. T1b creates both the original and catalogue tables; an empty folder can be opened without a schema error. Record both results and targets.		pass

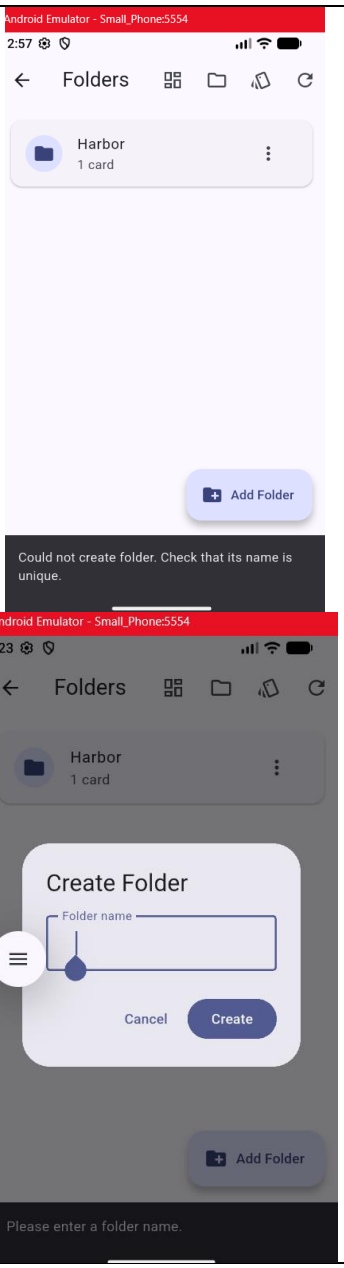
T2 / Parent-child create	On the upgraded target, create “Atlas” and “Harbor.” Add two cards titled “Compass” in Atlas and one card titled “Signal” in Harbor. Choose valid suits; leave Signal's image_ref null.	Three distinct card IDs; Atlas count 2, Harbor count 1; each card appears only under its folder. Record actual folder/card IDs.		pass

T3 / Edit identity	Edit only one Compass by ID to “Compass revised” with notes, and Save. Edit the other Compass, then Cancel.	Save returns 1 affected row; only the selected card changes. Cancel writes nothing; folder counts remain 2 and 1.		pass
T4 / Restart persistence	Capture the card IDs/counts, force-stop the app without clearing data, and relaunch.	Same folders, IDs, cards, values, and counts appear; no duplicate seed records.		pass

			 Android Emulator - Small_Phone:5554 12:45 Folders Atlas 2 cards Harbor 1 card Add Folder Android Emulator - Small_Phone:5554 12:45 Fall Festival Rost... Guest Name Age Add Guest Record Count: 3 Refresh 2 River ID: 2 • Age: 35 3 Acorn ID: 3 • Age: 0 4 Oak ID: 4 • Age: 130	

T5 / Card delete & cascade	Cancel deleting one Atlas card, then confirm that card deletion. Cancel deleting Atlas, then confirm the folder deletion.	Each Cancel changes nothing. Card delete returns 1; Atlas count becomes 1. Folder delete returns 1 parent row, removes the remaining Atlas card, and leaves Harbor/Signal intact. Confirm no card still references Atlas's ID.		pass

T6 / Validation & fallback	Open Signal with null image_ref. Try an Add with a whitespace-only title, a blank folder name, then duplicate folder name “Harbor,” separately.	Suit fallback displays for Signal. All invalid attempts give visible feedback and leave rows/counts unchanged. A forced read failure or network fault is not required.		pass

			 Android Emulator - Small_Phone:5554 2:57 Folders Harbor 1 card Add Folder Could not create folder. Check that its name is unique. Android Emulator - Small_Phone:5554 Folders Harbor 1 card Create Folder Folder name Cancel Create Add Folder Please enter a folder name.	